Carraux Guillaume, Bennani Maria, Perazzolo Sean

Bridging the Correctness-Security Gap:

A Formally Verified, Security-Aware Compiler Pipeline in Rocq

1. Introduction / problem statement

Mainstream compilers assume functional correctness is the only required property but in cryptographic software, this approach fails. Mathematically correct optimizations routinely delete security mechanisms (e.g., explicit memory clearing) via Dead Store Elimination (DSE) or introduce timing side-channels by optimizing constant-time logic into conditional branches. This is known as the "Correctness-Security Gap".

Inspired by the cryptographic language **Jasmin**, this project mathematically constrains compiler optimizations to respect some security properties.

2. Proposal

We will extend the `IMP` language from *Software Foundations* to build a formally verified, security-aware compiler pass. The project features a core Primary Objective, and a conditional Extension Objective if time permits.

2.1. Primary goal: Safe Memory Clearing via Dead Store Elimination

Our core goal tackles data persistence vulnerabilities.

- **Implementation:** We extend `IMP` with `Sensitive` variable markers. We add Liveness Analysis to safely eliminate non-sensitive dead variables for performance, but augment our Dead Store Elimination (DSE) pass to explicitly retain sensitive dead stores.
- **Verification:** Alongside functional correctness, we will prove a **Security Preservation Theorem**: the optimizer cannot eliminate dead stores marked sensitive. This guarantees the compiler preserves intentional memory clearing operations.

2.2. Extension goal: Secure Optimization of Constant-Time Branching

As an extension, we will tackle the preservation of constant-time execution.

- **Implementation:** We introduce a static Information Flow Type System (`Secret` vs `Public`) and a `CMOV` (Conditional Move) instruction. `CMOV` performs a conditional assignment without branching, ensuring execution time is independent of the condition's value.
- **Optimization Pass:** We implement a type-directed optimizer that converts `CMOV` instructions into faster `IF` branches **only** when the condition is statically typed `Public`. `Secret` conditions strictly remain `CMOV` instructions.
- **Verification:** By proving execution time trace equivalence for `Secret` variables, we mathematically guarantee the optimizer never introduces a timing side-channel.

3. Timeline and responsibilities

We leverage **Software Foundations** to avoid heavy infrastructure costs, distributing work evenly across three members. [TODO Precise task division]

We define three key milestones. Primary milestones (M1, M2) are sequential. Crucially, the Extension milestone (M3) is mathematically independent of M2, allowing parallel development of the Type System alongside the primary proofs.

- **W8**: Define core `IMP` extensions (`Sensitive` labels) and formalize basic evaluation relations.
- **W9: [Milestone 1]** Formalize the Liveness Analysis and assert functional correctness over the baseline extensions.
- **W10**: Implement the security-aware Dead Store Elimination compiler pass in Rocq.
- **W11: [Milestone 2]** Formulate and prove the Security Preservation Theorem concerning data persistence.
- **W12**: Introduce the Information Flow Type System and `CMOV` semantics.
- **W13: [Milestone 3]** Implement the type-directed optimizer (Objective 2), or dedicate this week strictly to resolving edge cases in the primary DSE proofs and polishing the codebase.
- **W14**: Draft, review, and submit the final academic report.